

# Sharding og MapReduce i MongoDB

---

## a) Hvad er sharding i MongoDB?

Sharding i MongoDB betyder, at vi splitter data op på flere servere, så databasen kan blive hurtigere, større og mere stabil. MongoDB bruger Shards, Config Server og Mongos for at få det til at fungere, uden at brugeren lægger mærke til det.

Det betyder, at vi deler ("sharder") data op i mindre bidder (kaldet shards) og fordeler dem på flere servere.

Eksempel:

- Tweets fra brugere med ID 1–1000 ligger på server A
- Tweets fra brugere med ID 1001–2000 ligger på server B
- osv.

## b) Hvilke komponenter kræves for at implementere sharding?

Der er tre vigtige komponenter i sharding:

- Shard: De servere, hvor data bliver gemt.
- Config Server: Holder styr på, hvor data ligger henne (en slags adressebog).
- Mongos: En slags "port", som brugerne forbinder sig til. Mongos finder ud af, hvor dataen skal hentes fra.

## c) Forklar arkitekturen af sharding i MongoDB

Når vi bruger sharding, sender brugeren sine forespørgsler til en mongos. Mongos snakker med config serveren for at finde ud af, på hvilken shard dataen ligger. Derefter henter den data fra den rigtige shard.

På den måde kan vi gemme kæmpestore databaser uden at alt ligger på én maskine.

## d) MapReduce – Implementering af Map- og Reduce-funktioner

Vi har brugt Map- og Reduce-funktioner til at finde hashtags i tweets.

```
var mapFunction = function() {  
  if (this.entities && this.entities.hashtags) {  
    this.entities.hashtags.forEach(function(tag) {  
      emit(tag.text.toLowerCase(), 1);  
    });  
  }  
};  
  
// Reduce function: lægger antallet sammen  
var reduceFunction = function(key, values) {  
  return Array.sum(values);  
};
```

Vi er stadig nye i MongoDB, men vi lærte, at emit() bruges til at sende noget ud i Map-fasen, og at Array.sum() samler resultaterne i Reduce-fasen.

## e) Kommando til at køre MapReduce

Vi kørte følgende kommando i MongoDB Shell:

```
db.tweets.mapReduce(  
  mapFunction,  
  reduceFunction,  
  {  
    out: "hashtags_count"  
  }  
);
```

Denne kommando opretter en ny collection, der hedder hashtags\_count, hvor alle hashtags og deres antal bliver gemt.

## Top 10 hashtags fra MapReduce-resultatet

Efter vi havde lavet MapReduce, brugte vi denne kommando for at finde de 10 mest populære hashtags:

*`db.hashtags_count.find().sort({ value: -1 }).limit(10).pretty();`*

---

```
twitterDB> db.tweets.mapReduce(
...   mapFunction,
...   reduceFunction,
...   {
...     out: "hashtags_count" // << this creates a new collection called hashtags_count
...   }
... );
...
DeprecationWarning: Collection.mapReduce() is deprecated. Use an aggregation instead.
See https://docs.mongodb.com/manual/core/map-reduce for details.
{ result: 'hashtags_count', ok: 1 }
twitterDB> db.tweets.mapReduce(
...   mapFunction,
...   reduceFunction,
...   {
...     out: "hashtags_count" // << this creates a new collection called hashtags_count
...   }
... );
... db.hashtags_count.find().sort({ value: -1 }).limit(10).pretty();
[
  { _id: 'nodejs', value: 29 },
  { _id: 'angularjs', value: 29 },
  { _id: 'fcblive', value: 27 },
  { _id: 'javascript', value: 22 },
  { _id: 'lfc', value: 19 },
  { _id: 'globalmoms', value: 19 },
  { _id: 'webinar', value: 18 },
  { _id: 'espanyolfcb', value: 18 },
  { _id: 'iwci', value: 17 },
  { _id: 'job', value: 13 }
]
twitterDB> |
```